Machine Learning Applications

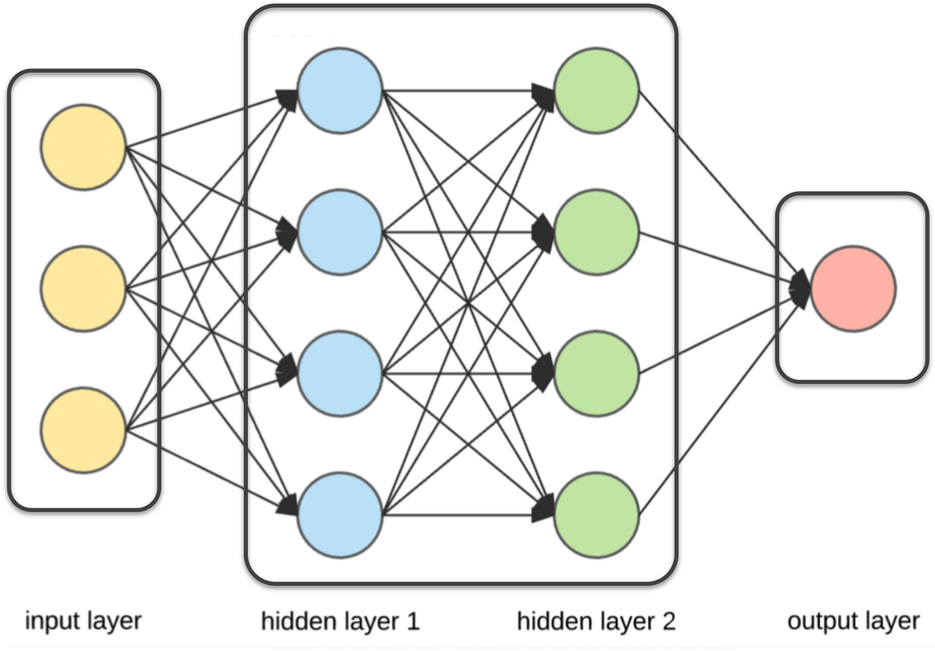
Neural Network

Introduction

Neural Network

Can use for classification & regression
Can consider when dealing with:
- Lots of data
- Lots of non-linear data
Hidden layers usually 2-3
Forward, fully-connected

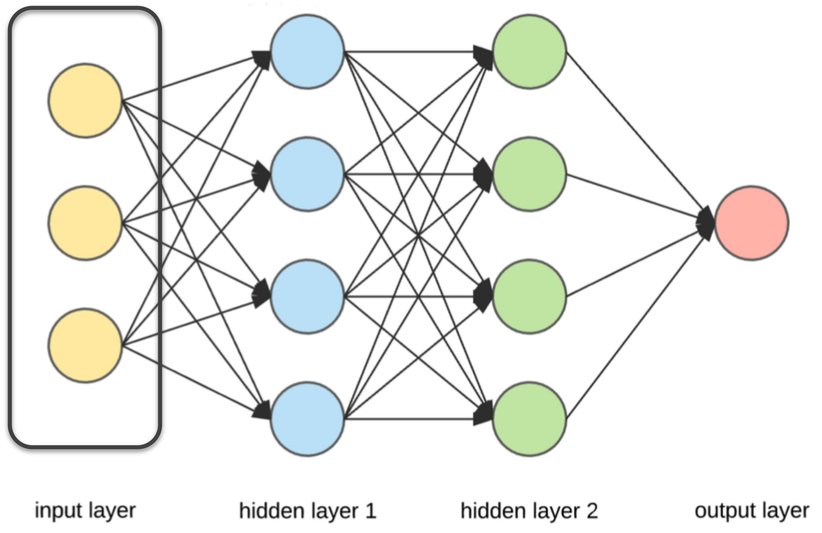
- A Neural Network is made of interconnected layers of computing units
- Common layers – Input, Hidden and Output



A Simple Neural Network

Input Layer

- The Input layer contains the inputs for computation
- If we are training a neural network to predict if a person is overweight, then the inputs could be Age, Weight and Height



A Simple Neural Network

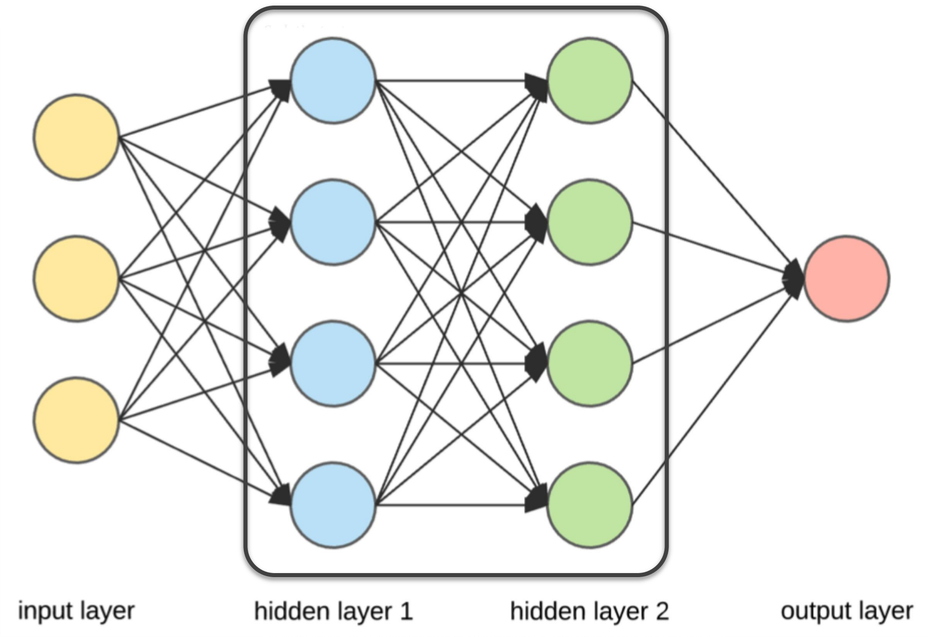
Hidden Layers

- Hidden layers are the layers between the Input and Output layers
- Bulk of computations happens in the Hidden layers

How to increase accuracy score?

1. Increase no. of neurons

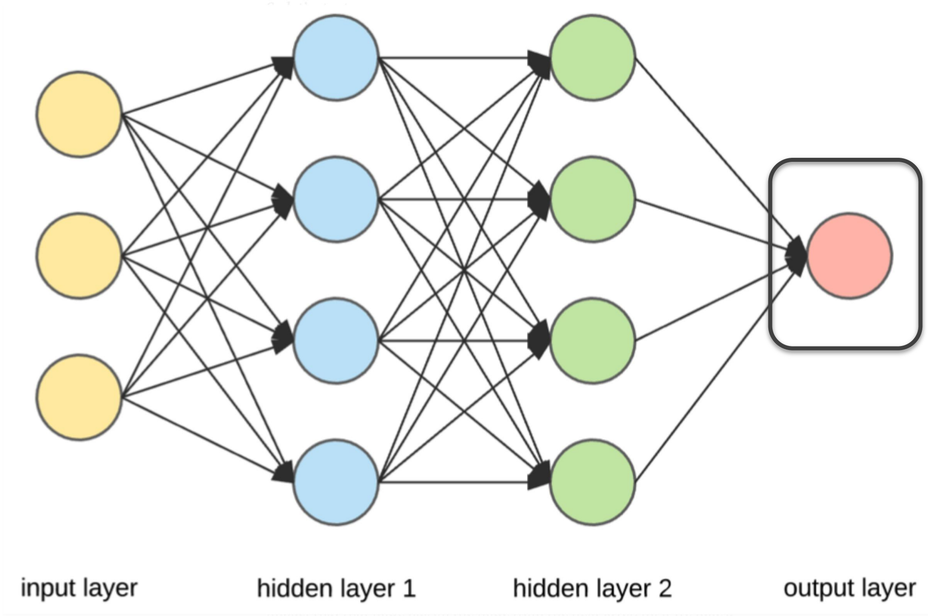
2. Increase hidden layers



A Simple Neural Network

Output Layer

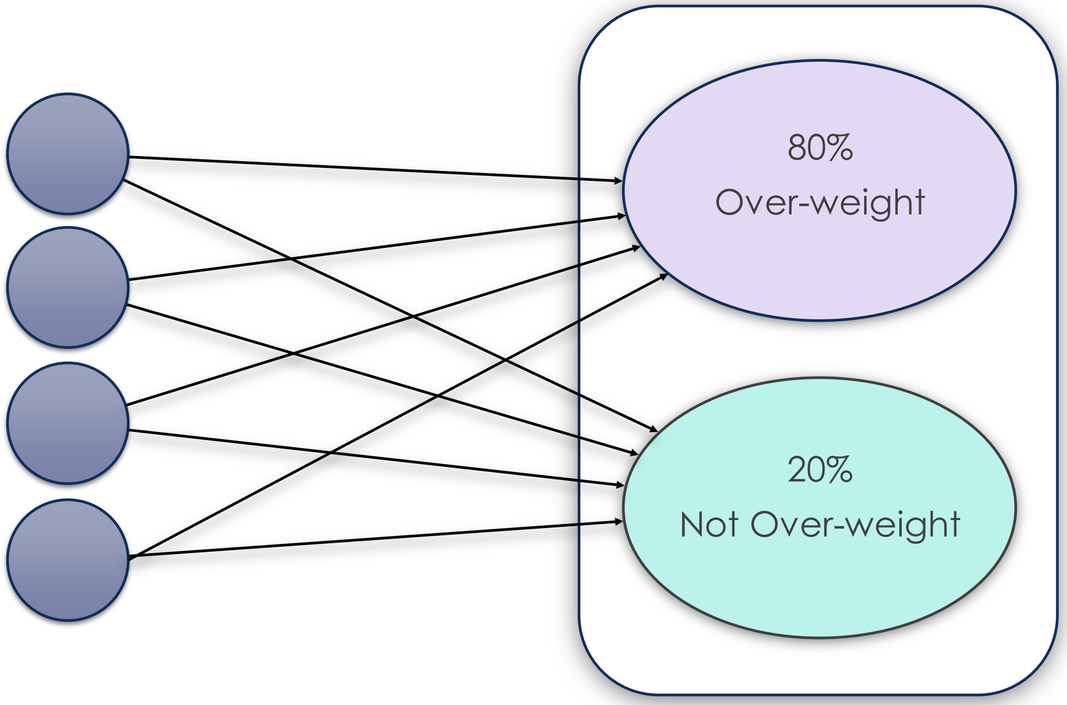
- The Output layer gives the computed prediction from a neural network
- For predicting if a person is overweight, the output can be a value between 0 and 1, where any value above 0.5 denotes the person is overweight



A Simple Neural Network

Output Layer

- Alternatively, our Output layer could have been designed with 2 output nodes in mind, where each node carries a probability of the person being overweight

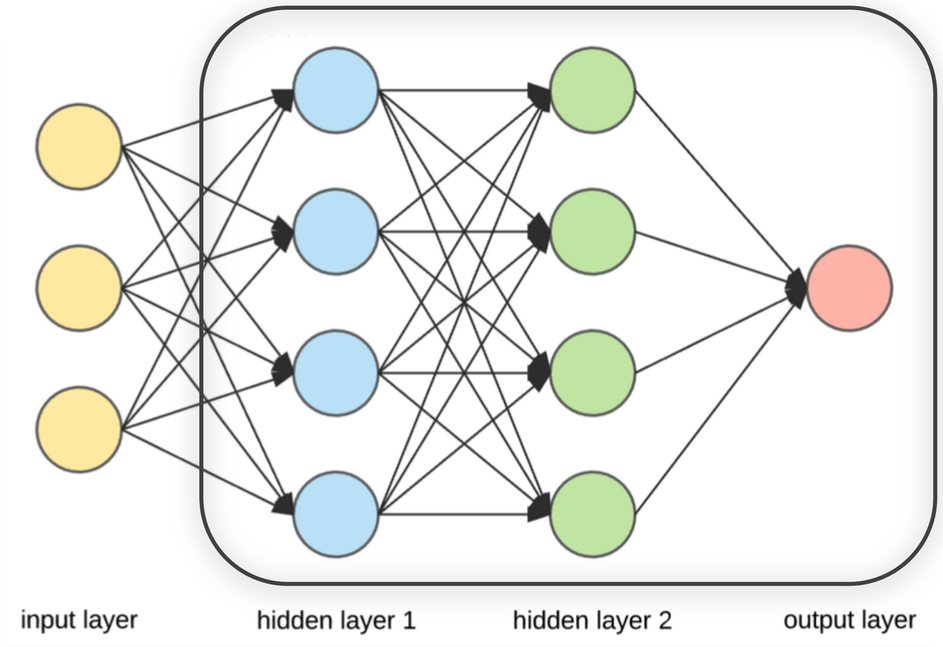


hidden layer 2

output layer (with 2 neurons)

A N-layer Neural Network

- A Neural Network is called N-layer if the sum of hidden and output layers is N
- The diagram depicts a 3-layer Neural Network



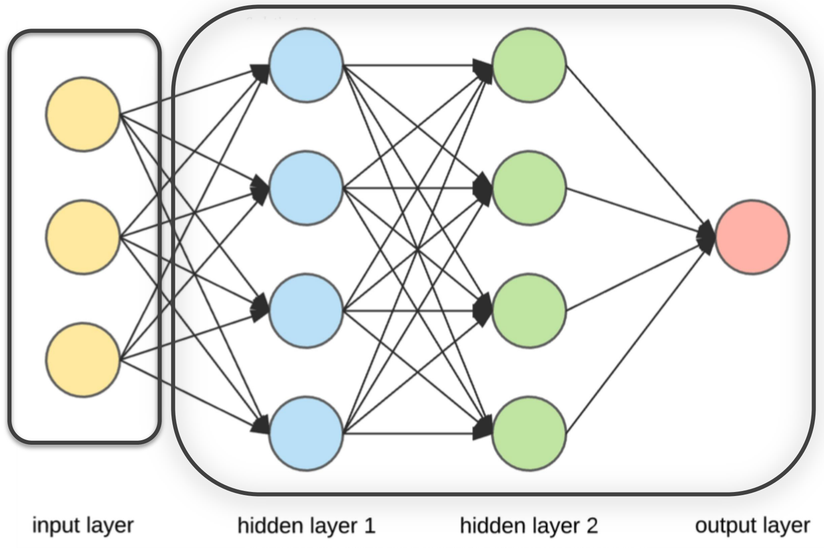
A Simple Neural Network

Neurons

- The nodes in a neural network are called neurons
- The **blue**, **green** and **red** nodes are neurons
- The yellow nodes are not neurons, they are just input values

Or nodes

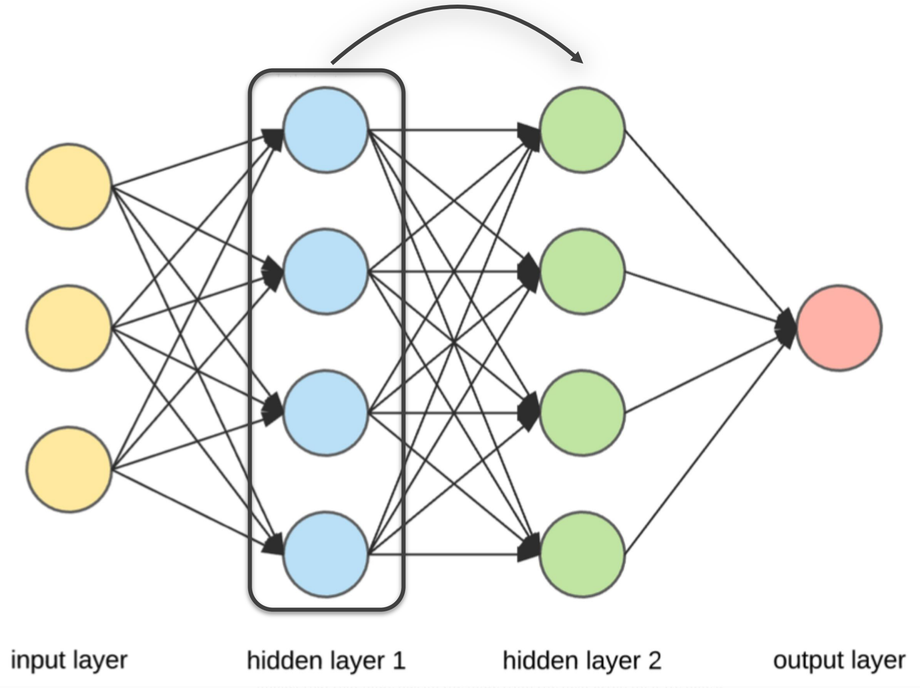
Input and output are computational



A Simple Neural Network

Neurons

- Each neuron contains mathematical functions
- Each neuron applies its mathematical functions on values that comes from a previous layer



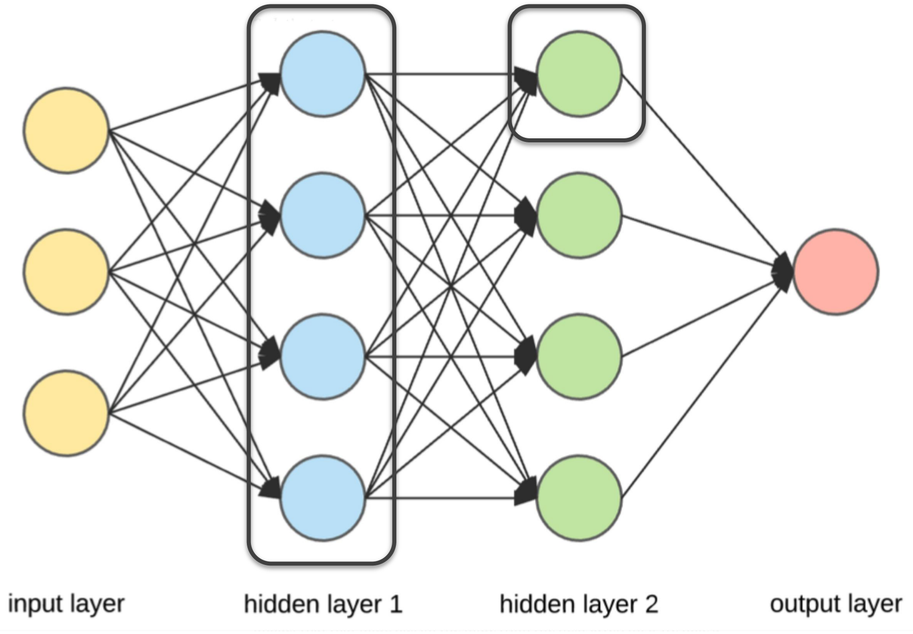
A Simple Neural Network

Neurons

- Each **green** neuron applies its mathematical functions on values coming from its connected 4 **blue** neurons in the previous layer

All arrows are forward, no backward;

Fully connectednetwork, meaning all nodes have connections; good for regression & classification

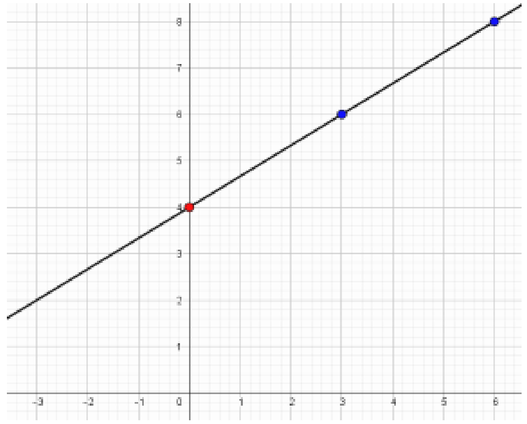


A Simple Neural Network

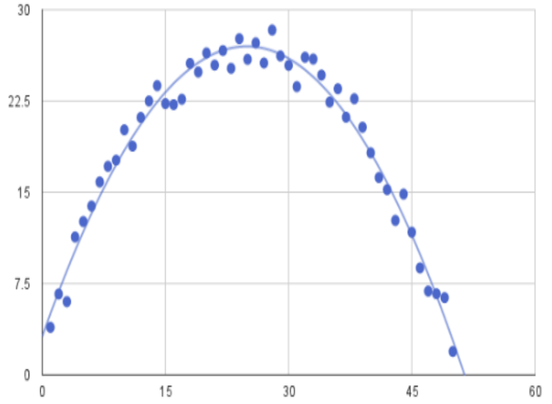
Weights and Biases

Universal Function Approximator

- Neural Networks are **Universal Function Approximators** that can approximate any mathematical function (linear or non-linear functions) As long as you have enough layers



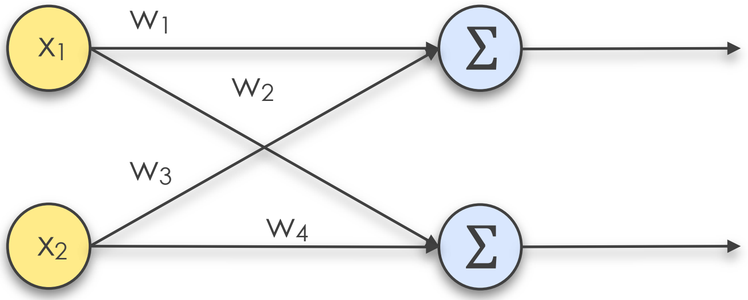
A linear equation



A non-linear equation

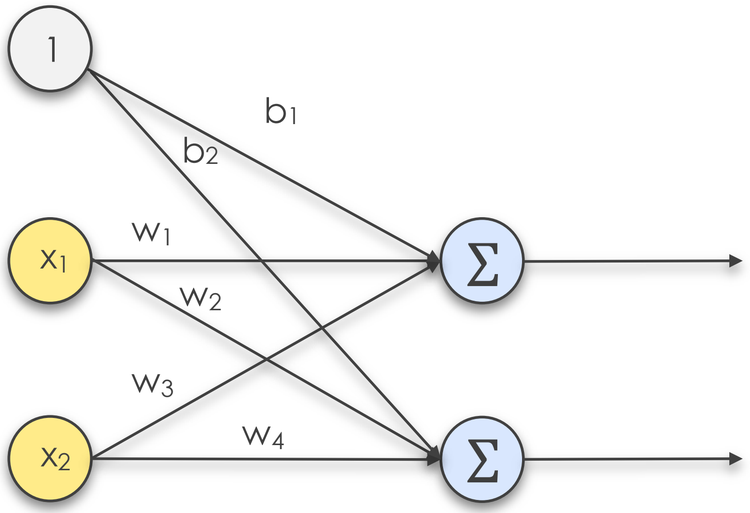
Weights

- Weights are numerical values attached to connections between neurons, and they control the impact of incoming values to the next neuron



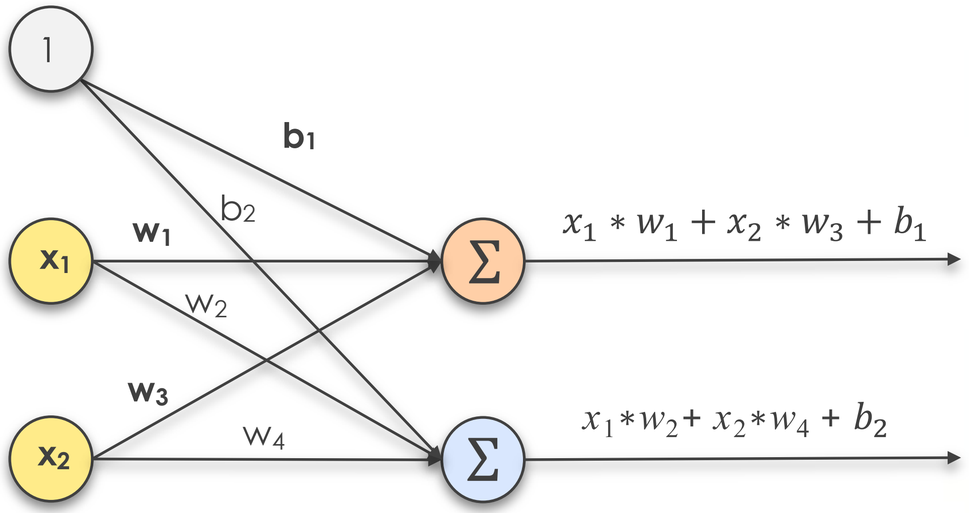
Biases

- Biases are special weights that allow finer adjustments to computed equations



Weighted Sum

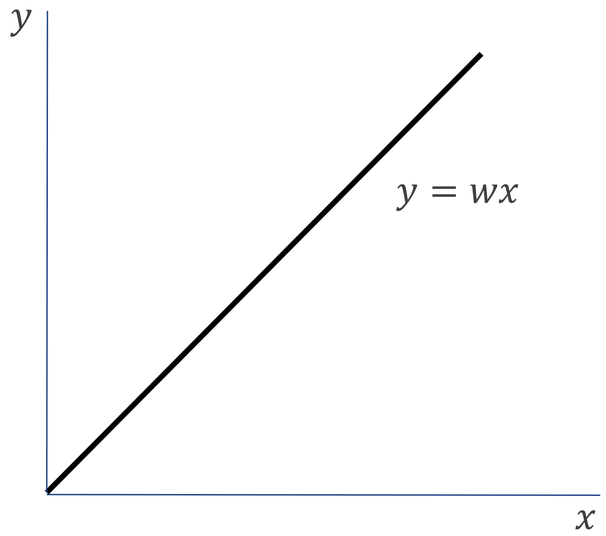
- Weighted Sums are computed by multiplying and adding weights and biases with inputs from previous layers



Weighted Sums calculation

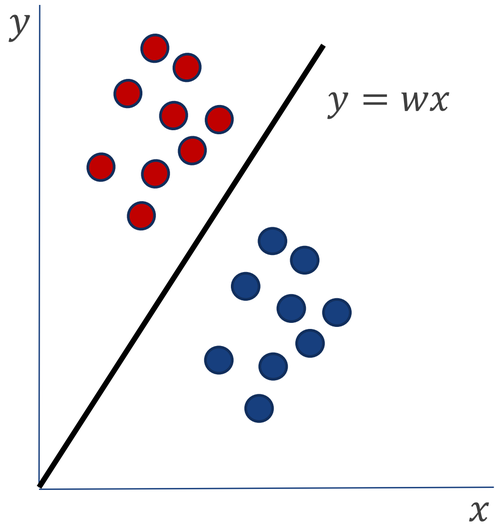
Weights only

- Let's look at a neural network that approximates a linear function that separates a set of data
- A straight line is defined as $y = wx$, where w is the gradient



Weights only

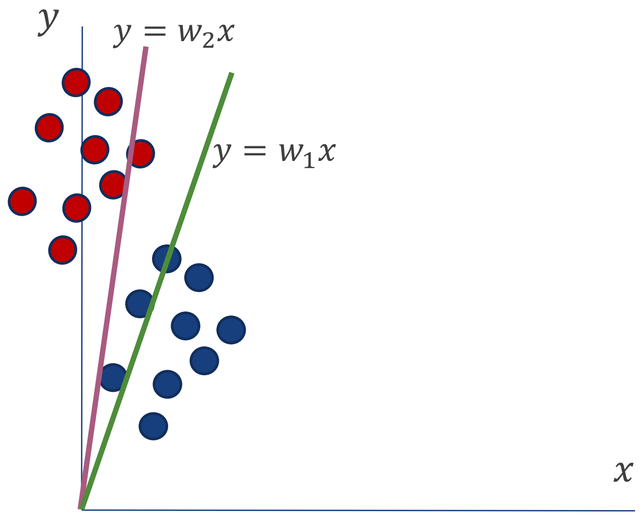
- We can only solve those classification problems whose data points can be separated by a line that intercepts the origin with Weights only



Weights only

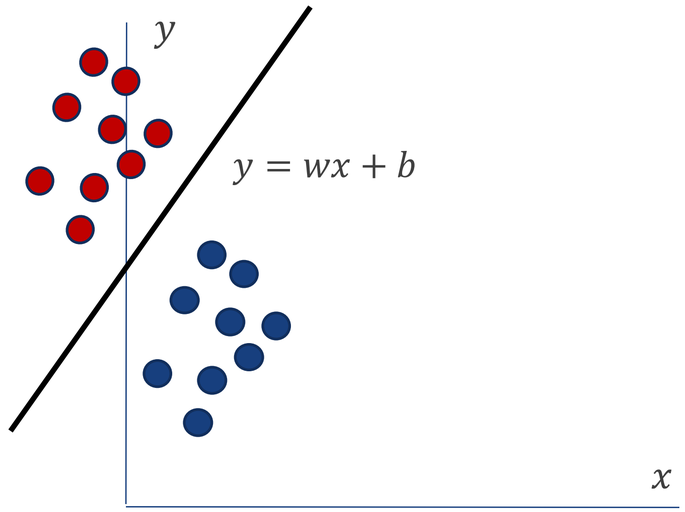
Weights only

- These data do not allow us to separate it cleanly by adjusting the gradient alone (i.e. using only Weights in our computation)



Weights and Biases

- Bias gives us the ability to move away from the origin
- The diagram shows the same problem solved by adding a bias

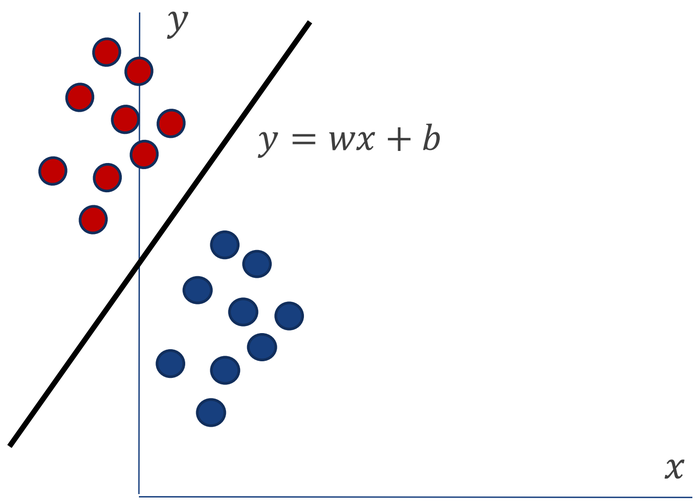


Weights and Biases

Activation functions

Weighted Sums only

- Linear solutions, such as the one provided by Weighted Sums, can solve problems that are linearly separable



Linearly separable data

Weighted Sums only

- A Weighted Sum operation yields a linear equation: $y = Wx + b$
- As the output of a weighted sum operation in one layer is the input to a weighted sum operation of the next, a neural network with only weighted sum operations can **only** produce **linear approximations**
- Example:

  $Y_1 = 2X + 1$

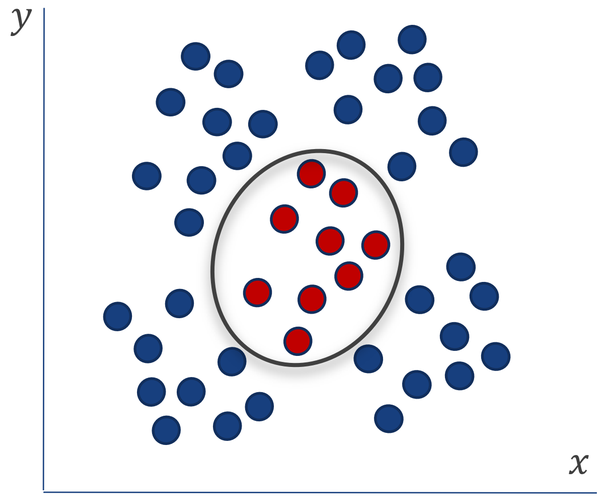
  $Y_2 = 3Y_1 + 4$

  $Y_2 = 3(2X + 1) + 4$

  $Y_2 = 6X + 7$ (still a linear equation)

Weighted Sums only

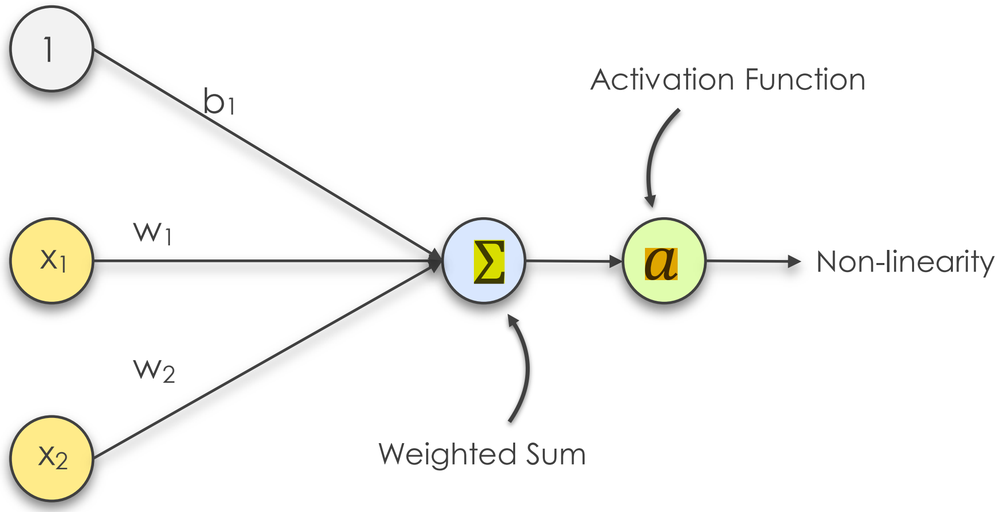
- Data that are non-linearly separable requires the use of **non-linear** equations
- We shall see how Activation Functions add non-linearity to our models to solve problems like such



Non-Linearly separable data

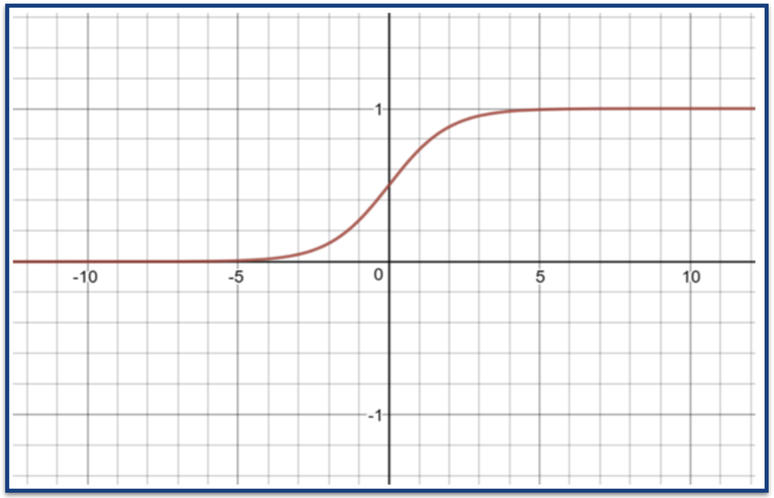
Non-linear transformation

- A Weighted Sum operation, followed by a Non-Linear Transformation applied by an **Activation Function**



Common Activation Function -Sigmoid

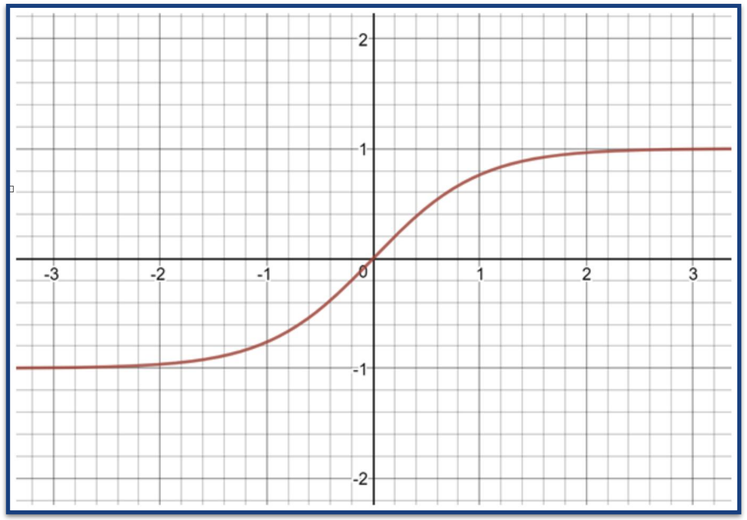
- The Sigmoid activation function, also called the Logistic function, transforms inputs into a value between 0 and 1
- Useful for models that predict a probability as an output



$$f(x) = \frac{1}{1 + e^{(-x)}}$$

Common Activation Function -Tanh

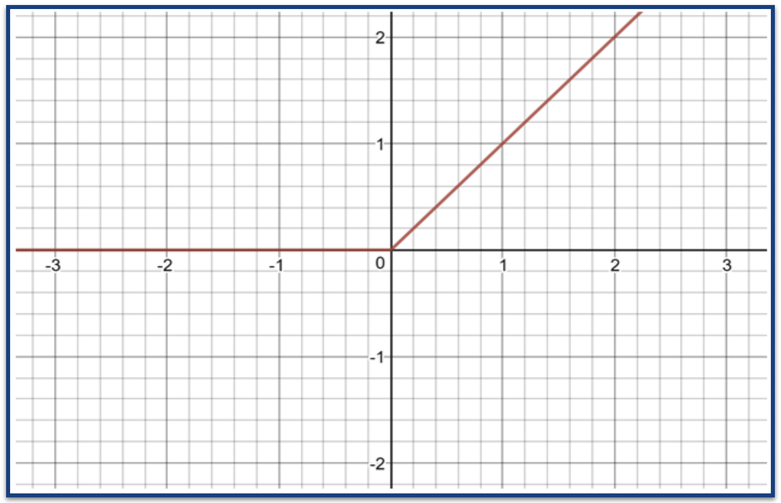
- The Tanh activation function transforms inputs to values between -1 and 1
- Useful for models that yields normalized positive and negative outputs



$$f(x) = \frac{2}{1 + e^{(-2x)}} - 1$$

Common Activation Function - ReLU

- The ReLU activation function transforms inputs with negative values to 0, but leave other values unchanged



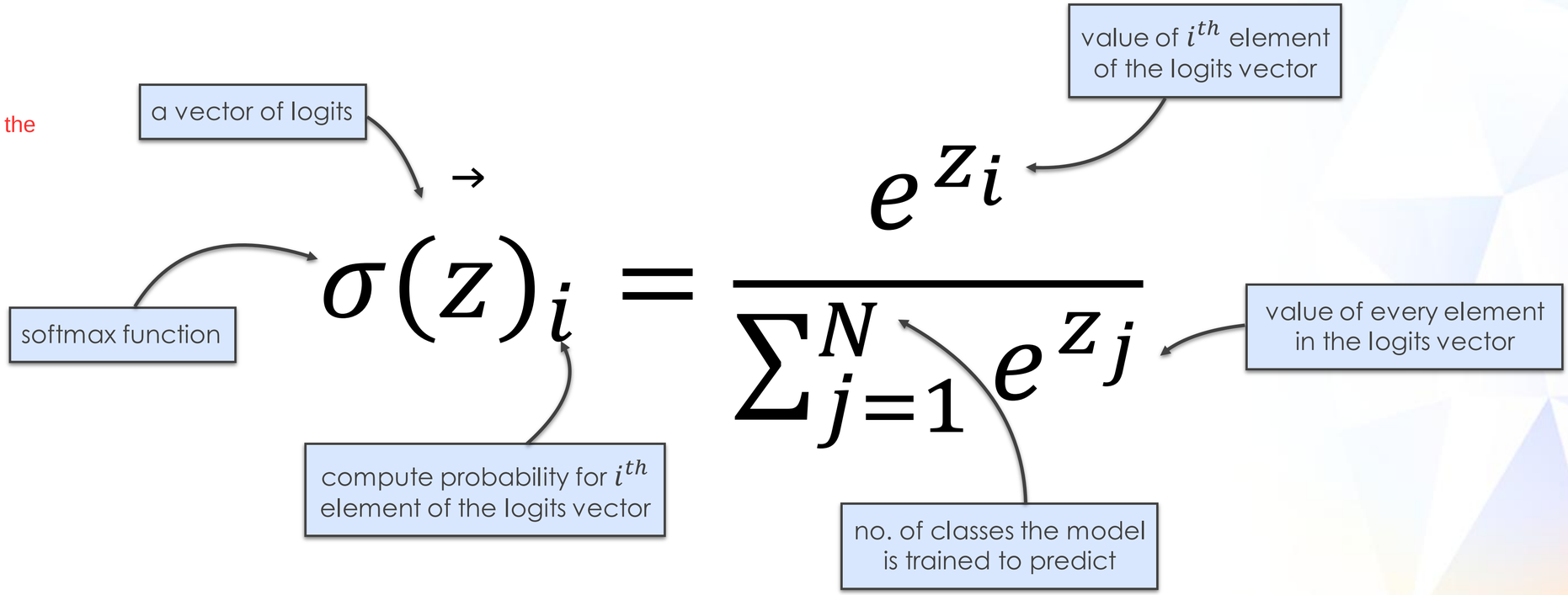
$$f(x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$$

Common Activation Function - Softmax

- The Softmax activation function takes a final computed vector and outputs a vector of probabilities as predictions

Applied on output layer

Tells you probability of the data



Summary of Activation Functions

- **Non-linearity**: Activation functions introduce non-linearity into the neural network, enabling the network to model complex relationships and patterns in the data.
- **Learning complex representations**: Without activation functions, the neural network would just be a series of linear transformations, limiting its ability to learn complex data structures.
- **Gradient-based learning**: Activation functions, particularly smooth ones like Sigmoid, Tanh, or Swish, allow for effective backpropagation, the algorithm used to optimize neural networks.

Loss Functions

Loss Functions

- When training, a **loss function** helps to **determine the error** in our model's prediction for a given sample
- The **error** tell us how far our **predicted value** is from the **actual value**
- Learning in neural network is simply minimizing a Loss Function
- Finding optimal values for parameters (weights and bias) to minimize the cost (Loss) function
- Loss functions are only used for **training** our models

Optimization Problem

- We need an optimization algorithm to train neural network

- We use the optimization algorithm to find the best values for weights and biases with minimizing the error

Loss Function

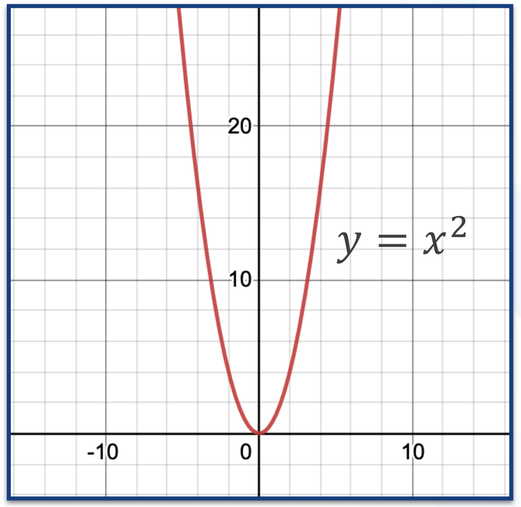
- Mean Square Error (aka L_2 loss) is a loss function that is commonly used

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Actual Value (Ground Truth)

Batch Size

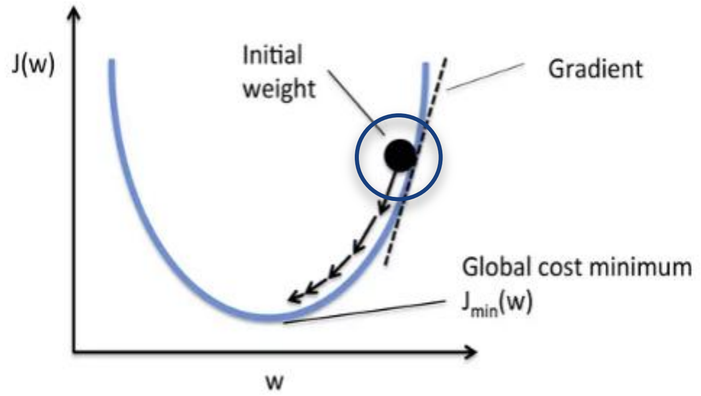
Predicted Value



- Note: except MSE, there are other loss functions, e.g. cross entropy, WCSS etc.

Gradient Descent

- Gradient Descent is an optimization technique to minimize a loss function by iteratively moving in the direction of the loss function's gradient
- It determines the amount of changes to be made to the Weights and Biases for a better prediction (i.e. smaller error) in the next iteration

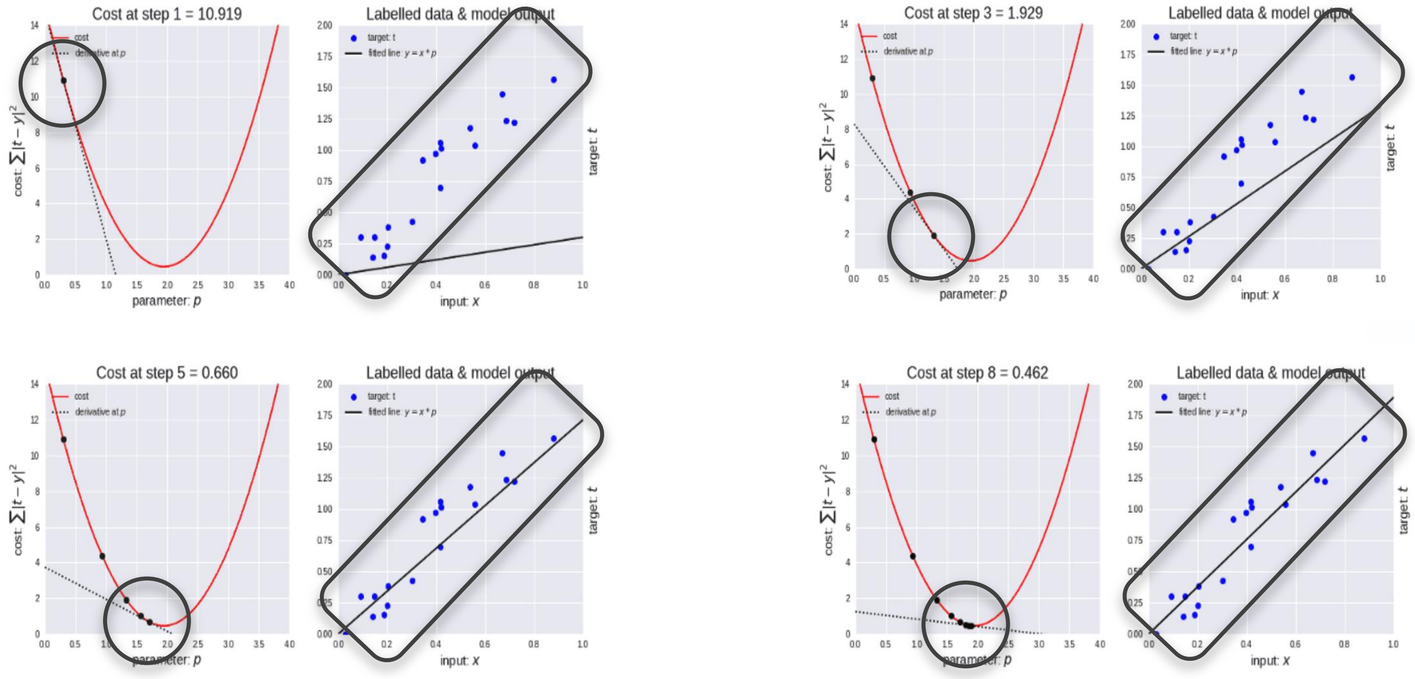


$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

η is the learning rate.

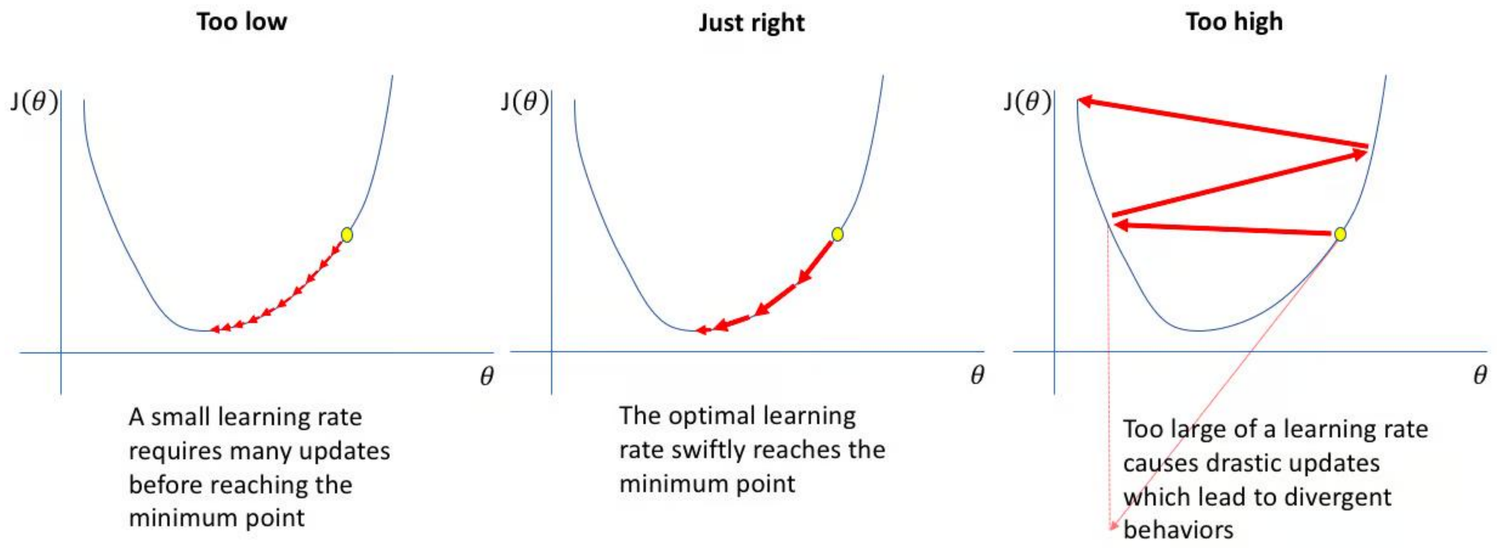
Loss Function

- An illustration to find a line that best fits the set of points by minimizing the MSE loss function



Learning Rate

- Learning rate is a hyper-parameter, it has an impact the final performance of the model.
- The learning rate η indicates the speed at which the coefficients evolve.
- It is essential to set learning rates to appropriate values to help the descent of slopes reach local minimums.



Fitting a Model

Data

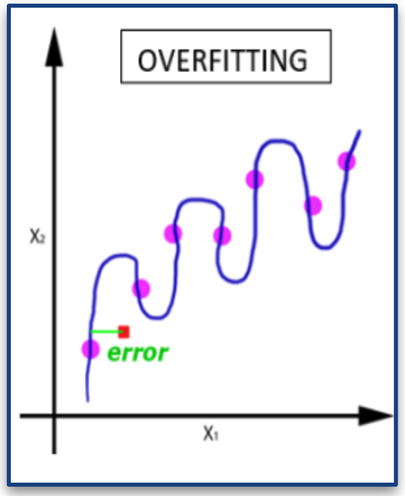
- Training data is used to **train** our model over multiple epochs
  - An **epoch** has occurred when our model has seen the **entire set** of data **once**
  - Our model **learns** during training; because its weights and biases are updated to make its prediction better

- Validation and Test data are used to access our model's accuracy
  - Validation data runs through the model once **at the end** of each **epoch**
  - Test data runs through the model once **at the end** of our **training**
  - Our model **does not learn** during validation or testing; because no weights and biases are updated

How many times you train using entire dataset; never use test data

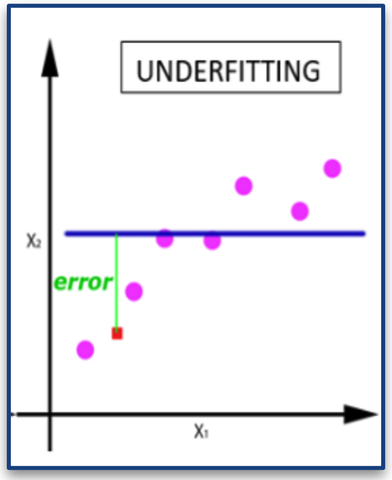
Over-Fitting

- If our model gives good predictions on training data, but predicts poorly on validation and/or test data, then it is **over-fitting** (it cannot generalize to data that it has not seen before)
- To resolve this, use more and better training data, or apply **regularization** (add an error term) during backpropagation



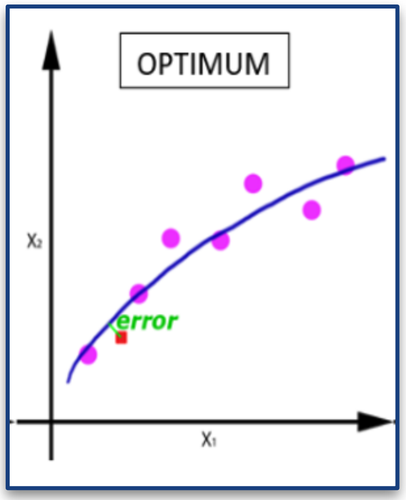
Under-Fitting

- If our model predicts poorly in training, it is **under-fitting**. The model is **not sophisticated enough** to learn the **underlying patterns** of the training data
- To resolve this, **train** our model **longer**, add **more hidden layers** (add sophistication) to it or feed it with **more** and/or **better** training data



Optimal Fitting

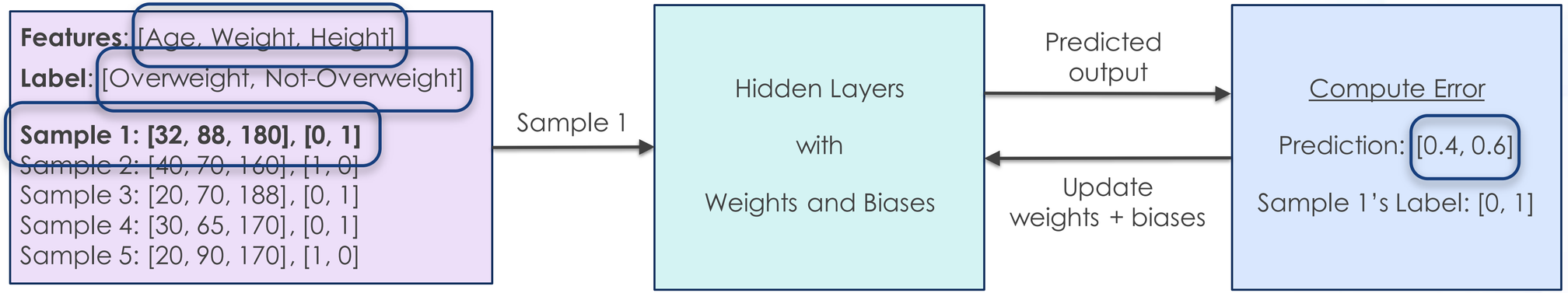
- If a model predicts correctly for its training, validation and/or test data, it has achieved **optimal fitting**
- It means our model can **generalize** for new/unseen data by learning from its training data



Training and Testing

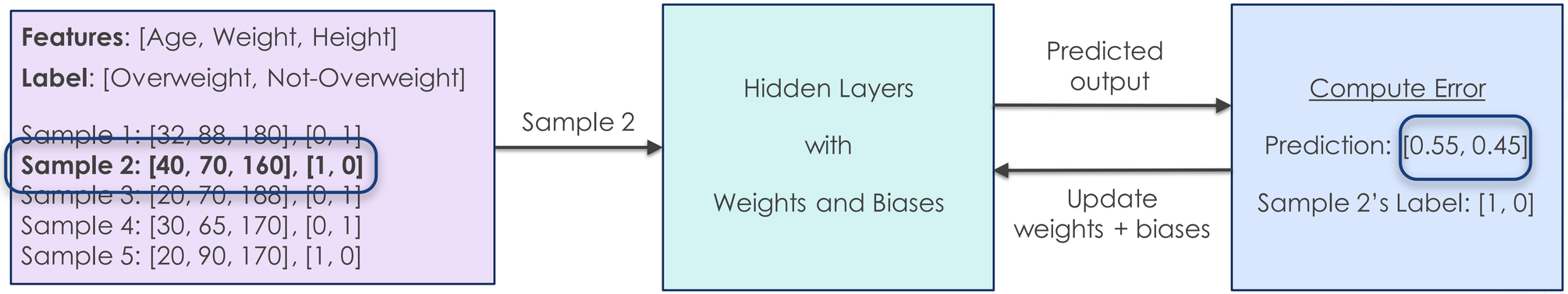
Training a Neural Network

- Train a Neural Network to predict if a person is overweight starting with Training Sample 1



Training a Neural Network

- Train a Neural Network to predict if a person is overweight using the next training-sample and so on



Using a trained Neural Network

- Using the trained Neural Network to predict if a person is overweight using Test Sample 1

Using a trained Neural Network

- Using the trained Neural Network to predict if a person is overweight using subsequent test samples

Neural Networks Applications

Neural Network for Regression – Add 3 Numbers

- Let's train a Neural Net to add 3 integers
- For example, if our Neural Net is given [1,2,3], it should output a value that is very close to 6
- The training data consists of arrays of 3 integers
- The labels are the sums of the 3 integers in each of the array
- Our Neural Net is solving a **Regression** problem

Add 3 Numbers (Training Data)

- Each sample, that consists of 3 integers, are randomly generated
- Each label is the sum of the 3 integers of that sample

```
import numpy as np
import pandas as pd
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
```

```
# ========================================
# 1. Generate Training Data
# ========================================
# Generate random integer: np.random.randint(low, high, size)
# generate data
def generate_data(low, high, n_rows, n_cols):
    x_data = np.random.randint(low, high, (n_rows, n_cols))
    y_data = np.sum(x_data, axis=1)
    return x_data, y_data
# new training and testing datasets
x_train, y_train = generate_data(-100, 100, 500, 3)
x_test, y_test = generate_data(-100, 100, 20, 3)
```

Add 3 Numbers (Architecture)

- Our network architecture consists of multiple layers, each layer smaller than the previous, and finally produces that 1 **output** for the predicted sum.

```
# ========================================
# 2. Build Neural Network
# 3 inputs -> first hidden layer with 16 neurons, activation function is relu ->
# second hidden layer with 8 neurons, activation function is relu-> output layer with one output
# ========================================

model = keras.Sequential([
    keras.Input(shape=(3,)),
    layers.Dense(16, activation='relu'),
    layers.Dense(8, activation='relu'),
    layers.Dense(1)
])
```

Add 3 Numbers (Compile Model)

- The code show the **Compile** for our model
  - Optimizer is adam
  - Loss function is mse
  - Metrics is rmse

```
# ======================================
# 3. Compile Model
# configure optimizer, loss function
# optimizer is adam, adam adapts learning individually for each parameter
# loss function is mse as it is a regression problem
# ======================================

model.compile(
    optimizer='adam',
    loss='mse',
    metrics=[keras.metrics.RootMeanSquaredError()]
)
```

Add 3 Numbers (Train Model)

- Train model using training dataset, epochs is 100

```
# ========================================
# 4. Train Model
# train NN using training data, epochs=100: training the entire dataset 100 times, verbose=1: progress bar shown
# ========================================

history = model.fit(
    x_train,
    y_train,
    epochs=100,
    verbose=1
)
```

Add 3 Numbers (Average Losses)

- The losses at each epoch is stored so that we can plot a **loss curve** of our training
- Take a look at the losses at each Epoch (Only select some for illustration)

```
Epoch 1/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 129.4171 - root_mean_squared_error: 11.3762
Epoch 2/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 81.0739 - root_mean_squared_error: 9.0041
Epoch 3/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 39.3763 - root_mean_squared_error: 6.2751
Epoch 4/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 12.4684 - root_mean_squared_error: 3.5311
Epoch 5/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 3.9634 - root_mean_squared_error: 1.9908
Epoch 6/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 2.3824 - root_mean_squared_error: 1.5435
Epoch 7/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 1.5356 - root_mean_squared_error: 1.2392
Epoch 8/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 1.0131 - root_mean_squared_error: 1.0065
Epoch 9/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 0.6687 - root_mean_squared_error: 0.8177
```

Add 3 Numbers (Evaluate Model with Test data)

- Checking the predicted values w.r.t our test data

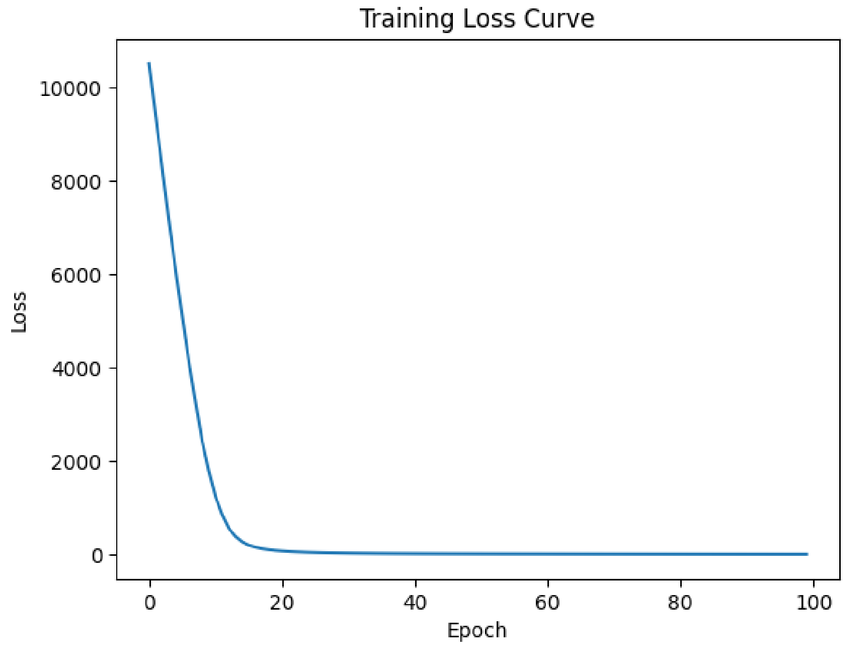
```
# evaluate model on test data
test_loss, test_rmse = model.evaluate(x_test, y_test, verbose=1)

print("Test Loss:", test_loss)
print("Test RMSE:", test_rmse)

# make predictions
predictions = model.predict(x_test)

# flatten prediction array
predictions = predictions.flatten()

# create tabular dataframe
results_df = pd.DataFrame({
    'x1': x_test[:, 0],
    'x2': x_test[:, 1],
    'x3': x_test[:, 2],
    'y': y_test,
    'predicted': predictions,
    'rounded_predicted': np.round(predictions)
})
```

	x1	x2	x3	y	predicted	rounded_predicted
0	83	47	-20	110	108.595810	109.0
1	73	82	-68	87	87.332359	87.0
2	1	-86	10	-75	-75.550102	-76.0
3	3	35	3	41	40.653660	41.0
4	-15	-48	69	6	8.769702	9.0
5	71	0	21	92	90.453850	90.0
6	-33	-16	55	6	9.553901	10.0
7	-71	-3	99	25	29.271969	29.0
8	-49	-19	15	-53	-52.434223	-52.0
9	16	43	35	94	96.019234	96.0
10	54	44	-55	43	47.222309	47.0
11	-86	-35	-81	-202	-205.465820	-205.0
12	-42	-42	-88	-172	-174.472229	-174.0
13	-61	63	-70	-68	-64.618668	-65.0
14	25	69	6	100	99.590347	100.0
15	-33	-50	-76	-159	-160.320267	-160.0
16	34	57	75	166	166.667999	167.0
17	17	28	-96	-51	-50.259289	-50.0
18	-40	-89	-88	-217	-216.463821	-216.0
19	-19	-63	-10	-92	-90.479530	-90.0

Add 3 Numbers (Loss Curve)

- Plotting the loss at each Epoch

```
# plot loss curve
plt.plot(history.history['loss'])

plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss Curve')

plt.show()
```



Neural Network for Classification - A Wine's Cultivar

- Let's train a Neural Network to determines a wine's cultivar
- The **features** are the attributes of the wine (e.g. Flavanoids, Hue)
- The **labels** are the class that a particular wine (sample) belongs to
- Our neural net is solving a **classification** problem

A Wine's Cultivar – Snapshots of the dataset

Features

Label

Cultivar	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline
1	14.23	1.71	2.43	15.6	127	2.8	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	13.2	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.4	1050
1	13.16	2.36	2.67	18.6	101	2.8	3.24	0.3	2.81	5.68	1.03	3.17	1185
1	14.37	1.95	2.5	16.8	113	3.85	3.49	0.24	2.18	7.8	0.86	3.45	1480
1	13.24	2.59	2.87	21	118	2.8	2.69	0.39	1.82	4.32	1.04	2.93	735
1	14.2	1.76	2.45	15.2	112	3.27	3.39	0.34	1.97	6.75	1.05	2.85	1450
1	14.39	1.87	2.45	14.6	96	2.5	2.52	0.3	1.98	5.25	1.02	3.58	1290
1	14.06	2.15	2.61	17.6	121	2.6	2.51	0.31	1.25	5.05	1.06	3.58	1295
1	14.83	1.64	2.17	14	97	2.8	2.98	0.29	1.98	5.2	1.08	2.85	1045
2	12.08	1.13	2.51	24	78	2	1.58	0.4	1.4	2.2	1.31	2.72	630
2	13.05	3.86	2.32	22.5	85	1.65	1.59	0.61	1.62	4.8	0.84	2.01	515
2	11.84	0.89	2.58	18	94	2.2	2.21	0.22	2.35	3.05	0.79	3.08	520
2	12.67	0.98	2.24	18	99	2.2	1.94	0.3	1.46	2.62	1.23	3.16	450
2	12.16	1.61	2.31	22.8	90	1.78	1.69	0.43	1.56	2.45	1.33	2.26	495
2	11.65	1.67	2.62	26	88	1.92	1.61	0.4	1.34	2.6	1.36	3.21	562
2	11.64	2.06	2.46	21.6	84	1.95	1.69	0.48	1.35	2.8	1	2.75	680
2	12.08	1.33	2.3	23.6	70	2.2	1.59	0.42	1.38	1.74	1.07	3.21	625
2	12.08	1.83	2.32	18.5	81	1.6	1.5	0.52	1.64	2.4	1.08	2.27	480
2	12	1.51	2.42	22	86	1.45	1.25	0.5	1.63	3.6	1.05	2.65	450
2	12.69	1.53	2.26	20.7	80	1.38	1.46	0.58	1.62	3.05	0.96	2.06	495
3	12.93	2.81	2.7	21	96	1.54	0.5	0.53	0.75	4.6	0.77	2.31	600
3	13.36	2.56	2.35	20	89	1.4	0.5	0.37	0.64	5.6	0.7	2.47	780
3	13.52	3.17	2.72	23.5	97	1.55	0.52	0.5	0.55	4.35	0.89	2.06	520
3	13.62	4.95	2.35	20	92	2	0.8	0.47	1.02	4.4	0.91	2.05	550
3	12.25	3.88	2.2	18.5	112	1.38	0.78	0.29	1.14	8.21	0.65	2	855
3	13.16	3.57	2.15	21	102	1.5	0.55	0.43	1.3	4	0.6	1.68	830
3	13.88	5.04	2.23	20	80	0.98	0.34	0.4	0.68	4.9	0.58	1.33	415
3	12.87	4.61	2.48	21.5	86	1.7	0.65	0.47	0.86	7.65	0.54	1.86	625
3	13.32	3.24	2.38	21.5	92	1.93	0.76	0.45	1.25	8.42	0.55	1.62	650
3	13.08	3.9	2.36	21.5	113	1.41	1.39	0.34	1.14	9.4	0.57	1.33	550
3	13.5	3.12	2.62	24	123	1.4	1.57	0.22	1.25	8.6	0.59	1.3	500

A Wine's Cultivar – Prepare Training and Testing data

- Perform Label Encoding for "Cultivar" column, convert label 1, 2, 3 to 0, 1, 2

as for loss function: `loss="sparse_categorical_crossentropy"` expected labels to be start from 0.

- Split dataset into training and testing datasets, 90% for training and 10% for testing
- **Straify** ensures that the proportions of the class-labels are preserved in both the train and test sets
- **PCA** is performed to **reduce the dimensions** of our data

```
# ================================
# 2. Split features and label
# ================================
X = df.drop("Cultivar", axis=1)

# use original Cultivar labels directly
y = df["Cultivar"]
# ================================
# 3. Perform Label Encoding
# ================================
encoder = LabelEncoder()

# convert labels into 0,1,2
y_encoded = encoder.fit_transform(y)

print("Original labels:")
print(y.unique())

print("Encoded labels:")
print(np.unique(y_encoded))
```

```
# ================================
# 4. Split dataset
# 10% testing
# ================================
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.10,
    random_state=42,
    stratify=y_encoded
)
```

```
# ================================
# 4. Standardize dataset
# ================================
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# ================================
# 5. Perform PCA
# ================================
pca = PCA(n_components=5)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Explained variance ratio:")
print(pca.explained_variance_ratio_)

print("Total variance captured:")
print(pca.explained_variance_ratio_.sum())
```

A Wine's Cultivar - Architecture

- The network architecture of our Neural Network
  - Input shape (5,)
  - First hidden layer with 16 neurons, activation function is relu
  - Second hidden layer with 8 neurons, activation function is relu
  - Output layer with 3 neurons (3 classifications), activation function is softmax

```
# ================================
# 6. Build Neural Network
# ================================
model = keras.Sequential([
    keras.Input(shape=(5,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(8, activation="relu"),
    layers.Dense(3, activation="softmax")
])
```

A Wine's Cultivar – Compile Model

- Loss function is `sparse_categorical_crossentropy` for multi-class integer labels
- Measure metrics is `accuracy`

```
# ================================
# 7. Compile model
# ================================
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

A Wine's Cultivar – Loss and Accuracy

- The loss and accuracy are captured for each epoch
- Total `epoch = 80`

```python
# ================================
# 8. Train model
# ================================
history = model.fit(
    X_train_pca,
    y_train,
    epochs=80,
    validation_data=(X_test_pca, y_test),
    verbose=1
)
```

```
Epoch 1/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 1s 70ms/step - accuracy: 0.3500 - loss: 1.1516 - val_accuracy: 0.4444 - val_loss: 0.9725
Epoch 2/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.3938 - loss: 1.0975 - val_accuracy: 0.4444 - val_loss: 0.9337
Epoch 3/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.4187 - loss: 1.0451 - val_accuracy: 0.4444 - val_loss: 0.8979
Epoch 4/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.4375 - loss: 0.9969 - val_accuracy: 0.5000 - val_loss: 0.8637
Epoch 5/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.4875 - loss: 0.9516 - val_accuracy: 0.5000 - val_loss: 0.8323
Epoch 6/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.5250 - loss: 0.9101 - val_accuracy: 0.5000 - val_loss: 0.8043
Epoch 7/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.5437 - loss: 0.8738 - val_accuracy: 0.6111 - val_loss: 0.7790
Epoch 8/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5813 - loss: 0.8395 - val_accuracy: 0.7222 - val_loss: 0.7568
Epoch 9/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6062 - loss: 0.8086 - val_accuracy: 0.7222 - val_loss: 0.7356
Epoch 10/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6313 - loss: 0.7784 - val_accuracy: 0.7778 - val_loss: 0.7159
Epoch 11/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.6562 - loss: 0.7503 - val_accuracy: 0.7778 - val_loss: 0.6977
Epoch 12/80
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.6750 - loss: 0.7243 - val_accuracy: 0.7778 - val_loss: 0.6804
```

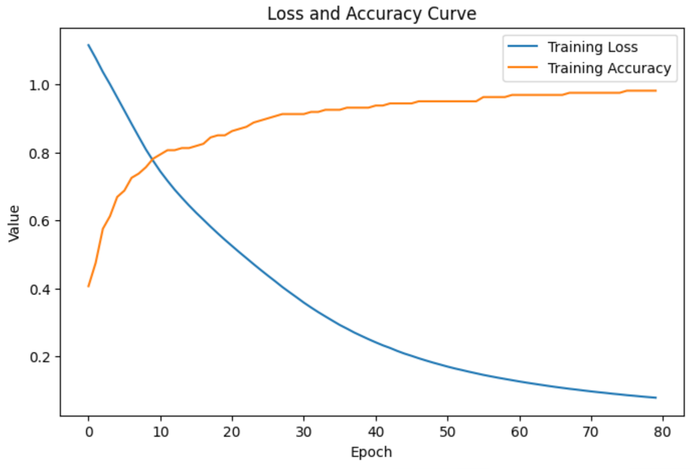
A Wine's Cultivar – Loss Curve

- Both loss and accuracy for training and testing datasets

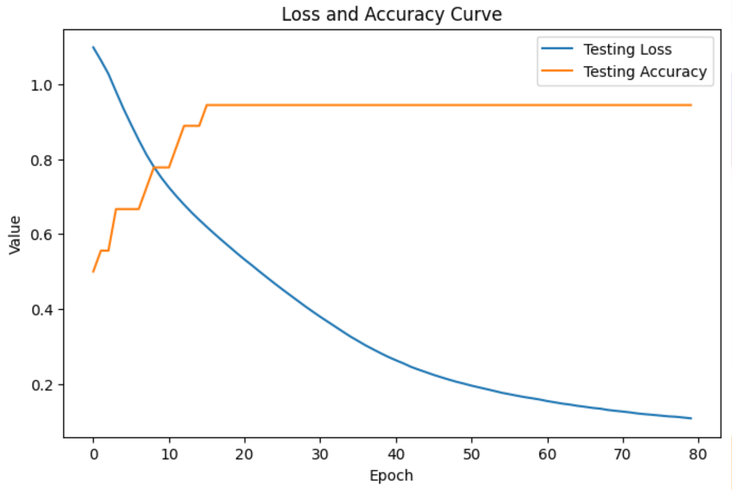
```
# ================================
# 9. Evaluate model
# ================================
test_loss, test_accuracy = model.evaluate(
    X_test_pca,
    y_test
)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
# 10. Plot loss and accuracy
# ================================
plt.figure(figsize=(8,5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Value")

plt.title("Loss and Accuracy Curve")

plt.legend()

plt.show()
plt.figure(figsize=(8,5))

plt.plot(history.history["val_loss"], label="Testing Loss")
plt.plot(history.history["val_accuracy"], label="Testing Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Value")

plt.title("Loss and Accuracy Curve")

plt.legend()

plt.show()
```

Training set



Testing set

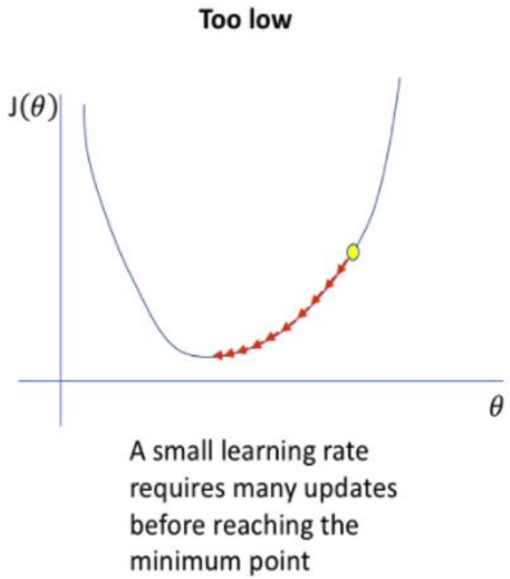


The End

Appendix (Optional)

Learning Rate

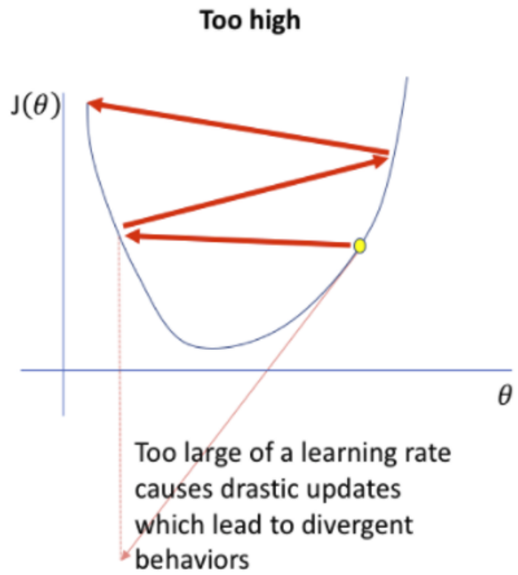
- The learning rate controls the amount of adjustments made to our weights and biases in each backpropagation
- A learning rate that is too small causes a neural network to learn slowly



Source: https://www.jeremyjordan.me/nn-learning-rate/

Learning Rate

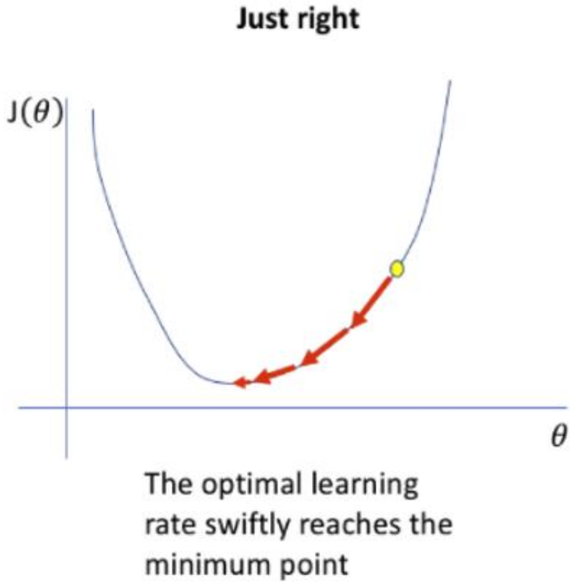
- A learning rate that is too large causes a neural network to yield losses that swing wildly from one iteration to the next



Source: https://www.jeremyjordan.me/nn-learning-rate/

Learning Rate

- An optimal learning rate allows a neural network to learn steadily and swiftly



The optimal learning rate swiftly reaches the minimum point

Source: https://www.jeremyjordan.me/nn-learning-rate/

Training dataset, Validation dataset and testing dataset

- In machine learning, the entire dataset is usually split into
  - Training dataset, validation dataset and testing dataset
  - Training dataset is used to training the model, learn weights/parameters
  - Validation dataset is used during training to evaluate intermediate performance, tune hyperparameters (learning rate, batch size, number of hidden layers), the model does not learn from validation data, no weights update occur.
  - Testing dataset is used only after training is completed.
- K-folder cross validation
  - Divide dataset into K equal folds. Train K times, each fold becomes the validation dataset once.

  Note: K-folder is more common in traditional ML and small dataset as deep learning training is expensive. DNN often uses train/validation/test split.